Decision Tree — Complete End-to-End Notes

1. Definition

A **Decision Tree** is a **supervised machine learning algorithm** used for both **Classification** and **Regression** problems. It works by splitting the dataset into smaller subsets based on feature values, forming a tree-like structure of decisions, ending in **leaf nodes** that represent the final prediction.

Structure:

- **Root Node** — represents entire dataset, first split
- **Decision/Internal Nodes** — represent a test on a feature
- **Branches** — outcome of the test (Yes/No, or value ranges)
- **Leaf Nodes** — final output (class label or numeric value)

Formal definition: A Decision Tree is a flowchart-like structure that recursively partitions the feature space into regions, and assigns a prediction to each region, by asking a sequence of if-else questions on the input features.

---

2. What Problems Can It Solve?

- **Classification**
    - Loan Approval (Yes/No)
    - Disease Diagnosis (Positive/Negative)
    - Customer Churn Prediction
    - Species classification (Iris dataset)
- **Regression**
    - House price prediction
    - Salary prediction
    - Sales forecasting

Key property: Works well for both **linear and non-linear relationships**, and it's the **foundation for ensemble models** like Random Forest, Gradient Boosting, XGBoost.

---

3. Why Use Decision Trees?

Reason	Explanation
Easy to interpret	"White box" model — can visualize and explain every decision
No feature scaling needed	Doesn't require normalization/standardization
Handles both numerical & categorical data	Very flexible
Captures non-linear relationships	Unlike Linear/Logistic Regression
Handles missing values	Some implementations handle this natively
Feature importance	Automatically ranks which features matter most
Non-parametric	No assumption about data distribution

---

4. Why Not Just Use Linear/Logistic Regression?

Limitation of Linear Models	How Decision Tree Solves It
Assumes linear relationship	Trees capture non-linear boundaries
Struggles with feature interactions	Trees naturally model interactions via splits
Sensitive to outliers	Trees are more robust (splits are rank-based)
Requires feature scaling	Trees don't need scaling
Can't handle categorical variables directly	Trees split categorical features naturally

But: Decision Trees have their own weakness — **high variance** (they overfit easily), which regression models don't suffer from as much.

---

5. Core Intuition — How Splitting Works

The tree tries to find **the best feature and threshold** at each node that best separates the data into pure groups (classification) or reduces variance (regression).

The big question at every node:

"Which feature and which split point creates the most 'pure' or 'homogeneous' child nodes?"

This "purity" is measured using **Entropy, Gini Impurity (classification)**, or **Variance/MSE (regression)**.

---

6. Mathematical Intuition — Classification Trees

A. Entropy (Measure of Impurity/Disorder)

$$Entropy(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Where:

- S= dataset at the node
- c= number of classes
- p_i= proportion of class $i$in the node

Key properties:

- Entropy = 0 → node is completely pure (all same class)
- Entropy = 1 → maximum impurity (50-50 split, for binary classification)
- Range: $[0, \log_2(c)]$

Example: Node has 10 samples: 6 "Yes", 4 "No"

$$p_{yes} = 0.6,\ p_{no} = 0.4$$
$$Entropy = -(0.6\log_2 0.6 + 0.4\log_2 0.4) = -(0.6 \times -0.737 + 0.4 \times -1.322)$$
$$= 0.442 + 0.529 = 0.971$$

Close to 1 → highly impure (near 50-50 split)

B. Information Gain (Used by ID3, C4.5)

Information Gain = Reduction in entropy after a split

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|\ S_v\ |}{|\ S\ |} \times Entropy(S_v)$$

Where:

- A= feature being tested

- S_v= subset of $S$where feature Ahas value v

Decision rule: At every node, choose the feature that gives **maximum Information Gain**.

Key property: Information Gain is **biased toward features with many unique values** (like ID columns) — this is a known weakness of ID3.

C. Gini Impurity (Used by CART — most common, used in sklearn)

$$Gini(S) = 1 - \sum_{i=1}^{c} p_i^2$$

Key properties:

- Gini = 0 → pure node
- Gini = 0.5 → maximum impurity (binary, 50-50 split)
- Computationally faster than entropy (no log calculation)

Example: Same node: 6 Yes, 4 No

$$Gini = 1 - (0.6^2 + 0.4^2) = 1 - (0.36 + 0.16) = 1 - 0.52 = 0.48$$

Gini Gain (used to select best split in CART)

$$Gini\ Gain = Gini(S) - \left[\frac{|\ S_{left}\ |}{|\ S\ |} Gini(S_{left}) + \frac{|\ S_{right}\ |}{|\ S\ |} Gini(S_{right})\right]$$

Choose split with **maximum Gini Gain** (i.e., maximum decrease in impurity).

D. Gain Ratio (Used by C4.5 — fixes IG's bias)

$$GainRatio(S, A) = \frac{IG(S, A)}{SplitInfo(S, A)}$$

$$SplitInfo(S, A) = -\sum_{v} \frac{|\ S_v\ |}{|\ S\ |} \log_2 \frac{|\ S_v\ |}{|\ S\ |}$$

This **penalizes features with too many distinct branches**, solving Information Gain's bias problem.

---

7. Worked Example (Step-by-Step Split)

Dataset: Predict "Play Tennis" (Yes/No) based on "Weather" (Sunny/Rainy)

Weather	Play Tennis
Sunny	Yes
Sunny	Yes
Sunny	No
Rainy	No
Rainy	No

Step 1: Root Entropy Total: 5 samples → 2 Yes, 3 No

$$Entropy(S) = -(0.4\log_2 0.4 + 0.6\log_2 0.6) = 0.971$$

Step 2: Split on Weather

Sunny subset (3 samples): 2 Yes, 1 No

$$Entropy(Sunny) = -(0.667\log_2 0.667 + 0.333\log_2 0.333) = 0.918$$

Rainy subset (2 samples): 0 Yes, 2 No

$$Entropy(Rainy) = -(0\log_2 0 + 1\log_2 1) = 0 \text{ (pure!)}$$

Step 3: Weighted Entropy After Split

$$Entropy_{split} = \frac{3}{5}(0.918) + \frac{2}{5}(0) = 0.551$$

Step 4: Information Gain

$$IG = 0.971 - 0.551 = 0.420$$

Since "Weather" is the only feature here, it becomes the root split. Rainy branch becomes a pure **leaf node → "No"**. Sunny branch needs further splitting (still impure).

---

8. Mathematical Intuition — Regression Trees

For regression, we don't use Entropy/Gini — we use **Variance Reduction** or **MSE**.

Variance / MSE at a node:

$$MSE(S) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2$$

Where $\bar{y}$= mean of target values at that node.

Variance Reduction (Splitting Criterion):

$$VarianceReduction = Var(S) - \left[\frac{|\ S_{left}\ |}{|\ S\ |}Var(S_{left}) + \frac{|\ S_{right}\ |}{|\ S\ |}Var(S_{right})\right]$$

Decision rule: Choose split that gives **maximum variance reduction** (i.e., children have lowest combined variance).

Prediction at leaf node (regression):

$$\hat{y}_{leaf} = \frac{1}{n}\sum_{i=1}^{n} y_i \text{ (mean of target values in that leaf)}$$

---

9. Tree Building Algorithm (General Steps)

1. **Start** with entire dataset at root node
2. For each feature, calculate the impurity measure (Gini/Entropy/Variance) for every possible split point
3. **Select best split** — the one that maximizes Information Gain / Gini Gain / Variance Reduction
4. **Split the data** into child nodes based on that feature/threshold
5. **Repeat recursively** for each child node
6. **Stop** when:
    - Node becomes pure (entropy/gini = 0)
    - Maximum depth reached
    - Minimum samples per leaf reached

- No further Information Gain possible

7. Assign **class label (majority vote)** or **mean value (regression)** to leaf nodes

This is called **Recursive Binary Splitting** — a **greedy, top-down algorithm** (it picks the best split at each step without looking ahead).

---

10. Popular Decision Tree Algorithms

Algorithm	Splitting Criterion	Tree Type	Handles
ID3	Information Gain	Classification only	Categorical features only
C4.5	Gain Ratio	Classification	Categorical + Numerical, handles missing values
CART	Gini Impurity (classification) / MSE (regression)	Both	Most widely used (sklearn default), binary splits only
CHAID	Chi-Square	Classification	Statistical significance based splits

---

11. Overfitting in Decision Trees — Why It Happens

A **fully grown tree** keeps splitting until every leaf is 100% pure → it **memorizes training data** (including noise) → **high variance, poor generalization**.

Symptoms:

- Very high training accuracy
- Poor test/validation accuracy
- Extremely deep tree with very few samples per leaf

---

12. Pruning — Solving Overfitting

A. Pre-Pruning (Early Stopping)

Stop tree growth early using constraints:

- max_depth — limit tree depth
- min_samples_split — minimum samples required to split a node
- min_samples_leaf — minimum samples required at a leaf

- max_leaf_nodes — limit total number of leaves
- min_impurity_decrease — minimum gain required to make a split

B. Post-Pruning (Cost Complexity Pruning)

Grow the full tree first, then **cut back branches** that don't improve generalization.

Cost Complexity Pruning formula (used in CART):

$$R_\alpha(T) = R(T) + \alpha \mid T \mid$$

Where:

- $R(T)$= total misclassification error (or MSE) of tree T
- $\mid T \mid$= number of leaf nodes (tree complexity)
- α= complexity parameter (regularization strength)

Key property: As αincreases, the tree gets pruned more aggressively (fewer leaves). We choose optimal αusing **cross-validation**.

---

13. Feature Importance in Decision Trees

Feature importance is calculated based on **total impurity reduction** contributed by each feature across all splits (weighted by number of samples at each node):

$$Importance(feature) = \sum_{\text{nodes using feature}} \frac{N_{node}}{N_{total}} \times \Delta Impurity$$

Higher value → more important feature in making predictions.

---

14. Handling Different Data Types

Data Type	How Decision Tree Handles It
Numerical	Finds optimal threshold (e.g., Age < 30) via sorting + testing midpoints
Categorical	Splits into subsets per category, or one-hot encodes (depends on implementation)
Missing values	C4.5 handles natively via probability weighting; sklearn requires imputation

---

15. Evaluation Metrics

For Classification Trees:

Same as Logistic Regression:

- Accuracy, Precision, Recall, F1-Score
- Confusion Matrix
- ROC-AUC

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

For Regression Trees:

- **MSE (Mean Squared Error)**

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- **RMSE**

$$RMSE = \sqrt{MSE}$$

- **MAE**

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- **R^2 Score**

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

---

16. Bias-Variance Tradeoff in Decision Trees

Tree Depth	**Bias**	**Variance**	**Result**
Shallow tree (underfit)	High	Low	Underfitting
Deep tree (fully grown)	Low	High	Overfitting
Optimal depth (pruned)	Balanced	Balanced	Good generalization

Key property: Decision Trees are **low-bias, high-variance** models by nature — this is exactly why ensemble methods (Random Forest, Bagging) were invented — to reduce variance by averaging many trees.

---

17. Advantages & Limitations

Advantages:

- Easy to understand & visualize (white-box model)
- No need for feature scaling
- Handles both numerical & categorical data
- Handles non-linear relationships naturally
- Automatic feature selection (importance ranking)
- Works for both classification & regression

Limitations:

- Prone to **overfitting** (high variance)
- **Unstable** — small changes in data can produce a completely different tree
- **Greedy algorithm** — doesn't guarantee globally optimal tree
- **Biased toward features with more levels** (in Information Gain)
- Struggles with **linear relationships** compared to linear models (needs many splits to approximate a diagonal boundary)
- Can create **overly complex trees** that don't generalize well without pruning

---

18. Key Hyperparameters (Practical Tuning)

Hyperparameter	**Purpose**
criterion	'gini' or 'entropy' (classification), 'mse'/'mae' (regression)
max_depth	Controls maximum depth of tree
min_samples_split	Minimum samples to allow a split
min_samples_leaf	Minimum samples required in a leaf node
max_features	Number of features considered for best split

ccp_alpha	Complexity parameter for cost-complexity pruning
max_leaf_nodes	Maximum number of leaf nodes allowed

---

19. Decision Boundary Behavior

- Decision Trees create **axis-parallel (rectangular) decision boundaries** — because each split is based on a single feature threshold (e.g., $x_1 < 5$)
- This means trees **struggle to model diagonal boundaries** efficiently (need many splits/steps to approximate a diagonal line)
- This is a key visual difference vs Logistic Regression, which creates a **single smooth linear boundary**

---

20. Quick Summary Table

Component	**Formula**
Entropy	$-\sum p_i \log_2 p_i$
Gini Impurity	$1 - \sum p_i^2$
Information Gain	$Entropy(S) - \sum\frac{$
Gain Ratio	$IG/SplitInfo$
Variance (regression)	$\frac{1}{n}\sum (y_i - \bar{y})^2$
Cost Complexity Pruning	$R(T)+\alpha$
Leaf Prediction (classification)	Majority class
Leaf Prediction (regression)	Mean of target values

---

21. Decision Tree vs Logistic Regression — Quick Comparison

Aspect	**Decision Tree**	**Logistic Regression**
Boundary type	Non-linear, rectangular	Linear (log-odds)
Interpretability	Very high (visual)	High (coefficients)
Feature scaling	Not required	Recommended
Overfitting risk	High (without pruning)	Lower
Handles feature interactions	Naturally	Needs manual interaction terms
Output	Class/Value	Probability
Bias-Variance	Low bias, high variance	Higher bias, lower variance